VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi-590018

Project report

on

WEATHER DATA STORAGE AND ANALYSIS USING MONGODB FOR BIG DATA PROCESSING

Submitted in partial fulfillment for the award of

Bachelor of Engineering

In

CSE-ARTIFICIAL INTELLIGENCE

Submitted by

DHANUSH JADAV
(1VE23CA014)

Under the guidance of

Mrs. Vimala Devi R

Assistant Professor

Dept. of CSE (AI), SVCE

SVCE | SRI VENKATESHWARA COLLEGE OF ENGINEERING

ESTD. 2001. AUTONOMOUS INSTITUTE

Department Of Computer Science Engineering (Artificial Intelligence)

Sri Venkateshwara College of Engineering, Bengaluru – 562157

2025 - 2026

ABSTRACT

Weather observations are generated continuously and are useful for studying temperature, rainfall, humidity, wind, and seasonal patterns. This project develops a Python-based workflow for collecting historical observations, validating them, and storing them in a local MongoDB Community Server database. It is designed to provide a structured, repeatable way to query weather records from multiple Indian cities.

The implementation supports hourly historical data retrieval from the Open-Meteo Historical Weather API and incremental import of compatible CSV files. Pandas reads CSV data in chunks, while normalization routines parse timestamps, sanitize meteorological values, reject invalid records, and map accepted rows to a common document schema. PyMongo performs unordered bulk upserts into MongoDB collections, with indexes to support filtering and prevent duplicate logical observations.

MongoDB aggregation pipelines calculate summaries and trends without first loading the full collection into a Python data frame. A Streamlit dashboard presents these results through Plotly charts and provides city and date filters, data exploration, ingestion controls, database diagnostics, and CSV export. MongoDB Community Server is the storage and query engine; MongoDB Compass is an optional graphical client for inspecting it.

The supplied project contains automated tests for data validation, API response parsing, CSV handling, database upsert logic, aggregation behavior, and application imports. All 19 tests passed during this review. The supplied sample CSV contains five hourly Bengaluru observations. Live database totals and the previously reported dashboard figures could not be independently confirmed because the MongoDB service was inaccessible in this environment.

TABLE OF CONTENTS

1. 1. INTRODUCTION 1
2. 2. SYSTEM REQUIREMENTS AND SPECIFICATIONS 3
3. 3. SYSTEM DESIGN 5
4. 4. IMPLEMENTATION 7
5. 5. RESULT 12
6. 6. CONCLUSION 14

1. 1. INTRODUCTION

Weather conditions affect transport, agriculture, water management, public health, and everyday planning. Historical observations help analysts compare conditions across locations and time periods. When records are stored in separate files or examined manually, it becomes difficult to maintain consistent formats, prevent duplicates, and answer questions across large date ranges.

This project addresses those needs with a local document database and a Python analytics application. MongoDB stores each observation as a flexible document containing location, timestamp, granularity, source, and meteorological measurements. This structure supports records from the historical API and compatible CSV datasets while allowing analytical queries to filter and group the documents.

The system uses Python for orchestration and validation, PyMongo for database access, Pandas for chunked CSV processing, and Streamlit with Plotly for interactive presentation. The data source implemented for historical retrieval is the Open-Meteo Archive API. The code also includes a registry of ten Indian cities. The project is intended as an educational analytics system and does not issue forecasts or safety alerts.

Problem Statement: A practical way is needed to collect and organize historical weather observations from different sources, validate their fields, store them with duplicate protection, and produce useful summaries without manually processing the full dataset for every query.

Project Scope: The implementation covers hourly API retrieval, CSV ingestion, canonical record validation, local MongoDB storage, compound indexes, aggregation-based analytics, and a Streamlit dashboard. The API fetcher divides long date ranges into bounded requests. CSV processing reads chunks to limit memory use. Both ingestion paths share the same record schema and database upsert layer.

The project focuses on historical data analysis rather than real-time sensor acquisition or weather forecasting. Open-Meteo provides archived weather series; compatible CSV data can also be imported when its fields can be mapped to the supported schema. MongoDB Compass may be used to browse the database, but it is not required to run the Python application and does not replace the MongoDB server.

The system is useful because it keeps data ingestion, storage, analysis, and presentation in one reproducible workflow. A consistent schema simplifies city and date filters. Server-side aggregation allows the database to perform grouping and summary calculations. The dashboard gives users a visual interface for exploring the results and exporting filtered records.

The objectives are to acquire historical observations from supported sources; validate timestamps and meteorological values; store records in MongoDB with indexes and idempotent upserts; compute temperature, precipitation, humidity, wind, extremes, and data-completeness summaries; and present interactive charts and filtered records in a dashboard.

The expected benefit is an extensible foundation for comparative weather analysis. It demonstrates how a document database can support mixed-source time-series records and how database aggregation can reduce unnecessary transfer of raw records into the application. The reported results in this document distinguish tested code and supplied sample data from figures that could not be checked against a live database.

2. 2. SYSTEM REQUIREMENTS AND SPECIFICATIONS

The application runs on a Windows development computer and connects to a local MongoDB Community Server. Hardware guidance below is a practical recommendation for running the database, Python environment, and interactive dashboard together; it is not a measured configuration of the project computer.

MongoDB Compass is optional and serves as a graphical database client. The required database service is MongoDB Community Server. Python package versions are managed through requirements.txt; the project does not pin exact versions.

Hardware Requirements

No.	Component	Minimum	Recommended
1	Processor	64-bit dual-core CPU	Modern quad-core CPU
2	Memory	8 GB RAM	16 GB RAM
3	Storage	10 GB free for software and data	SSD with additional dataset capacity
4	Operating system	Windows 10 or later, 64-bit	Windows 11, 64-bit
5	Network	Internet for API retrieval	Stable broadband connection

Software Requirements

No.	Software	Role / requirement
1	Python 3	Runs ingestion, validation, analytics, and dashboard modules
2	MongoDB Community Server	Required local database service
3	PyMongo	Python driver for MongoDB
4	Pandas	Chunked CSV ingestion and tabular handling
5	Streamlit	Interactive web dashboard
6	Plotly	Interactive charts
7	Requests / python-dotenv	HTTP requests and environment configuration
8	MongoDB Compass	Optional GUI client for database inspection

Default System Configuration

Setting	Configured value
MongoDB URI	mongodb://127.0.0.1:27017/
Database	weather_analytics
Observation collection	weather_observations
Run collection	ingestion_runs
Connection timeout	5000 ms default
Bulk batch size	1000 default

Historical source	Open-Meteo Archive API
Sample data	5 Bengaluru hourly rows, 1 Jan 2023

The configuration defaults point to mongodb://127.0.0.1:27017/ and the weather_analytics database. Environment variables can override the URI, database name, connection timeout, and batch size. Collection and index setup is defined in db.py and is performed by the application connection workflow.

3. 3. SYSTEM DESIGN

The design separates ingestion, validation, persistence, analytics, and presentation. Historical API requests are made by the fetcher; CSV uploads are handled by the importer. Both paths normalize records before calling the shared PyMongo bulk-upsert layer.

The canonical weather_observations collection stores individual observations. The ingestion_runs collection records run metadata and processing counts. The MongoDB server executes aggregation pipelines for summaries, city comparisons, time trends, precipitation, extremes, data quality, and bounded relationship samples. The Streamlit application converts returned results into tables and Plotly visualizations.

The main components and data flow are:

1. Historical Open-Meteo Archive API
2. Compatible CSV file, processed in chunks with Pandas
3. Python validation and normalization

PyMongo batch upserts to local MongoDB Community Server

MongoDB collections, indexes, and aggregation pipelines

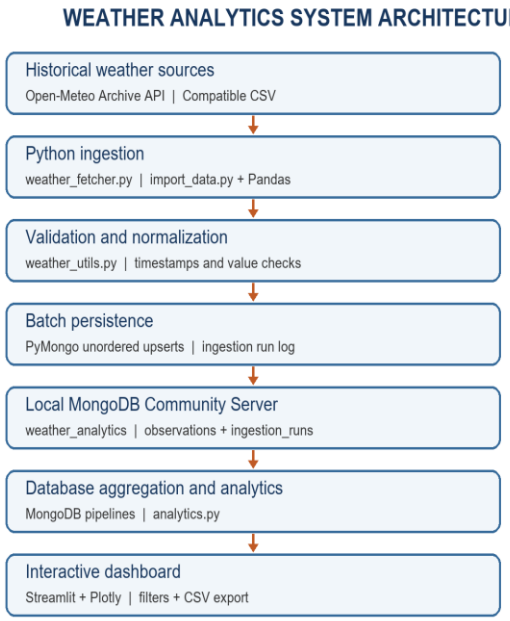
Python analytics functions and Streamlit interface

Plotly charts, filtered records, diagnostics, and CSV export

The architecture provides the following behavior:

- A common schema maps multiple sources to the same fields.
- Timestamp and numeric checks reject malformed or out-of-range values.
- Compound unique keys support repeatable upserts for a city, timestamp, granularity, and source.
- MongoDB performs grouping and summary work close to stored data.
- The dashboard applies city, date, granularity, and source filters where available.
- Compass is available for manual inspection but is not part of the storage execution path.

The unique observation index uses city, timestamp, granularity, and source. Additional indexes support timestamp, city-time, and granularity queries. Ingestion metadata is maintained separately so an import can be audited without mixing run-level details into every observation.



4. 4. IMPLEMENTATION

The project is implemented as a set of Python modules. config.py contains environment-driven MongoDB settings, the Open-Meteo endpoint, supported API variables, and the Indian city registry. weather_fetcher.py retrieves and parses API responses. weather_utils.py validates and normalizes records. import_data.py handles CSV input. db.py manages MongoDB connections, indexes, bulk upserts, and ingestion-run records. analytics.py defines aggregation queries, and app.py provides the Streamlit dashboard. The software dependencies listed in requirements.txt include PyMongo, Pandas, Streamlit, Plotly, Requests, python-dotenv, Pytest, and mongomock. MongoDB Community Server must be installed and running locally for database operations. Compass can be installed separately when visual inspection is desired.

4.1 Environment and Configuration

The application reads MONGODB_URI, MONGODB_DATABASE, MONGODB_CONNECT_TIMEOUT_MS, and MONGODB_BATCH_SIZE from the environment, with defaults in config.py. The default URI is local loopback port 27017 and the default database is weather_analytics. A connection check is implemented in check_connection.py.

Configuration steps:

1. Install Python and the packages listed in requirements.txt.
2. Install MongoDB Community Server and start its service.
3. Optionally install MongoDB Compass as a client for inspecting records.
4. Set environment variables when the default URI or database should be changed.
5. Run the application scripts from the project directory.
   The repository README documents a virtual environment setup and dashboard launch command.

4.2 Historical API Ingestion

weather_fetcher.py requests hourly variables from the Open-Meteo Historical Archive endpoint. The implementation maps temperature, relative humidity, precipitation, wind speed and direction, surface pressure, cloud cover, and weather code into canonical field names.

API processing workflow:

1. Select registered cities and a valid inclusive start and end date.
2. Split long date ranges into bounded chunks.

3. Request each chunk and parse the returned hourly arrays.
4. Normalize observations with location, timezone, source, and granularity metadata. Record run status and accepted, rejected, inserted, or updated counts when the database operation completes.

4.3 CSV Ingestion and Data Validation

import_data.py reads CSV files with Pandas chunks, maps recognized column names to the common schema, and allows a default city where the file does not provide one. It supports a dry-run option for validation without database writes. The supplied sample_weather.csv contains five rows for Bengaluru, dated 1 January 2023 from 00:00 through 04:00.

Validation and normalization include:

1. Parse supported timestamp representations and normalize timestamps to UTC-aware values.
2. Require a city and valid timestamp; reject records that fail required-field checks.
3. Sanitize numeric meteorological values against bounds defined in weather_utils.py.
4. Normalize the granularity to hourly or daily and assign a source label.
5. Preserve missing measurements as null values instead of converting them to zero.
6. Process rows incrementally so a large input file need not be loaded as one data frame.
7. The importer reports read, accepted, rejected, inserted, and updated record counts.

The example CSV is a small fixture and should not be interpreted as the full dashboard dataset.

4.4 MongoDB Schema and Indexes

The weather_analytics database uses two collections. weather_observations stores one normalized weather observation per document. ingestion_runs stores source, requested locations and date range, run times, status, and processing counts.

Observation fields include:

- city, country, latitude, longitude, timezone, timestamp, granularity, and source.
- temperature_c, relative_humidity_pct, precipitation_mm, wind_speed_kmh, and wind_direction_deg.
- pressure_msl_hpa, cloud_cover_pct, weather_code, and ingested_at.
- A compound unique index on city, timestamp, granularity, and source supports idempotent upserts.
- Additional timestamp, city/timestamp, and granularity indexes support common filters.

The separate ingestion_runs collection provides an audit history of import activity.

4.5 Bulk Write and Duplicate Prevention

db.py converts normalized documents into unordered bulk update operations. Each operation matches the compound logical key and uses upsert behavior. Re-ingesting the same source record therefore updates the matching document instead of intentionally adding a second copy. The unique index enforces the same key at the database level.

The ingestion workflow:

1. Prepare a bounded batch of validated documents.
2. Build a filter from city, timestamp, granularity, and source.
3. Submit unordered bulk operations through PyMongo.
4. Collect inserted and updated counts from the bulk result.
5. Handle bulk write and connection errors through the database layer.

---

6. Write or update an ingestion run record with the final status and counts.
7. Refresh dashboard data after a successful interactive ingestion.
8. Use the unique index to reject duplicate logical observations.

The implementation is designed for repeatable imports and bounded batch processing.

4.6 Aggregation and Analytics

analytics.py builds MongoDB aggregation pipelines for database summaries and analytical views. The queries filter records, group by city or time period, project derived values, and sort results before returning bounded data to the dashboard.

Implemented analyses include:

- Counts, date coverage, temperature extrema, averages, and precipitation totals.
- Temperature summaries by city and daily or monthly temperature trends.
- Annual comparisons, monthly rainfall trends, and wettest dates.
- Humidity summaries, weather-code distributions, and high-wind records.
- Data-quality completeness and null-field summaries.

  A bounded sample for variable relationships and Pearson correlation displays.

4.7 Dashboard and Data Export

The Streamlit application provides eight navigation areas: overview; temperature trends; multi-city comparison; precipitation; wind and extremes; variable relationships; data explorer; and database diagnostics and ingestion. Plotly charts visualize returned aggregates. Filters select cities and date ranges, and the data explorer supports paginated database queries and CSV export.

Dashboard functions supported in source:

- Overview KPIs and city summaries.
- Daily and monthly temperature charts.
- Comparisons for temperature, rainfall, humidity, and wind.
- Precipitation analysis, high-wind thresholds, and temperature extremes.
- Correlation displays with a note that association does not establish causation.

  Database health, collection counts, data completeness, and API/CSV ingestion controls.

5. RESULT

The source code and supplied data demonstrate a complete implementation path from historical API or CSV input to MongoDB storage, aggregation, and dashboard presentation. The test suite completed with 19 passing tests during this review. These tests cover representative logic paths with mocked responses and in-memory database fixtures; they do not prove that an external API request or a live MongoDB deployment succeeded during this review.

The supplied sample CSV has five valid-looking hourly rows for Bengaluru on 1 January 2023. It includes temperature, humidity, precipitation, wind, pressure, cloud cover, and weather-code columns. The test suite includes a separate importer fixture that checks accepted and rejected rows in dry-run mode.

The application code contains dashboard views for temperature, precipitation, city comparison, wind events, extremes, relationships, data exploration, and database diagnostics. The configured Open-Meteo fetcher supports ten registered Indian cities. These are implemented capabilities visible in source code, not independently confirmed live data results.

A previous dashboard view was reported to display 26,280 observations, a mean temperature of 24.89 °C, cumulative precipitation of 3,704.1 mm, a January-to-December 2023 coverage interval, and Bengaluru, Delhi, and Mumbai. The MongoDB service was not reachable from this environment, and no dashboard capture was available for inspection; these figures are therefore recorded as previously reported dashboard values, not verified database measurements.

No processing-speed benchmark or live collection size is claimed. The five-row CSV fixture is not representative of the 26,280 observations mentioned above. MongoDB Compass screenshots and weather dashboard screenshots were not available in the supplied project files, so the legacy installation screenshots were removed and the system architecture figure was replaced with a project-specific diagram.

The evidence supports the implemented workflow and unit-level behavior. A live deployment result would require MongoDB Community Server to be running and the intended dataset to be ingested. The dashboard figures should be refreshed from the database before being presented as independently measured results.

The system is suitable as a demonstrator for historical data ingestion and analytics. Its practical value depends on source-data quality, database availability, and the chosen date range. Interpretations of correlations remain exploratory and should not be used as causal conclusions or weather forecasts.

6. CONCLUSION

This project implements a local weather data storage and analytics workflow using Python and MongoDB. It provides historical API retrieval, chunked CSV import, normalization and validation, PyMongo bulk upserts, MongoDB indexes and aggregation pipelines, and a Streamlit and Plotly dashboard for interactive analysis.

The system stores observations separately from ingestion-run metadata and uses a compound unique key to support duplicate-resistant imports. The analytics layer includes city and time-period summaries, rainfall and wind analysis, extremes, data completeness, and bounded variable relationships. The dashboard also provides filtered record exploration and CSV export.

During this review, all 19 automated tests passed and the supplied CSV was confirmed to contain five Bengaluru hourly records. The MongoDB service could not be accessed from this environment, so collection counts and earlier dashboard metrics remain unverified. The report treats those earlier figures as reported dashboard values rather than measured database results.

Possible future improvements include validating a full production-sized dataset, adding reproducible ingestion logs, documenting a live deployment and database backup procedure, and extending the dashboard with carefully sourced comparisons. Any future performance claims should be based on recorded benchmark conditions and repeatable measurements.

The implementation demonstrates how a document database and server-side aggregation can organize historical weather observations for interactive analysis, while preserving a clear distinction between implemented features, tested behavior, and results that require live data verification.